

Two Pointers Cheat Sheet (Python)

Core pattern: move two indices toward each other (opposite ends) or together (fast/slow) based on a comparison — avoids the nested loop

1. Two Sum II (Sorted Array)

Pointers: left, right ends **Move rule:** sum vs target

```
def twoSum(numbers, target):
    left, right = 0, len(numbers) - 1
    while left < right:
        s = numbers[left] + numbers[right]
        if s == target:
            return [left + 1, right + 1]
        elif s < target:
            left += 1
        else:
            right -= 1
    return []
```

Time: O(n) **Space:** O(1)

Follow-ups

- Unsorted array version — how does the approach change?
- Return all pairs summing to target (skip duplicates).
- What if you need the lexicographically smallest pair?
- Generalize to k numbers summing to target (k-Sum).

2. Valid Palindrome

Pointers: left, right ends **Skip:** non-alnum chars

```
def isPalindrome(s):
    left, right = 0, len(s) - 1
    while left < right:
        while left < right and not s[left].isalnum():
            left += 1
        while left < right and not s[right].isalnum():
            right -= 1
        if s[left].lower() != s[right].lower():
            return False
        left += 1
        right -= 1
    return True
```

Time: O(n) **Space:** O(1)

Follow-ups

- Ignore only spaces, keep punctuation — what changes?
- Return the cleaned/filtered string too.
- Handle a streaming input (can't index from the end).
- Make the alnum check Unicode-aware.

3. Valid Palindrome II

Pointers: left, right ends **Budget:** 1 deletion

```
def validPalindrome(s):
    def isPal(i, j):
        while i < j:
            if s[i] != s[j]:
                return False
            i += 1
            j -= 1
        return True
    left, right = 0, len(s) - 1
    while left < right:
        if s[left] != s[right]:
            return isPal(left+1, right) or isPal(left, right-1)
        left += 1
        right -= 1
    return True
```

Time: O(n) **Space:** O(1)

Follow-ups

- Generalize to "at most k deletions allowed."
- Return the resulting palindrome string.
- Count the number of ways to fix it with one deletion.
- What if deletions are only allowed from the ends?

4. Container With Most Water

Pointers: left, right ends **Move:** shorter wall

```
def maxArea(height):
    left, right = 0, len(height) - 1
    best = 0
    while left < right:
        h = min(height[left], height[right])
        best = max(best, h * (right - left))
        if height[left] < height[right]:
            left += 1
        else:
            right -= 1
    return best
```

Time: O(n) **Space:** O(1)

Follow-ups

- Why is moving the shorter wall always safe?
- Return the actual pair of indices for the best container.
- Find the top-3 largest areas instead of just the max.
- Extend to 3 lines — is a greedy two-pointer still optimal?

5. Trapping Rain Water

Pointers: left, right ends **State:** left_max, right_max

```
def trap(height):
    left, right = 0, len(height) - 1
    left_max = right_max = water = 0
    while left < right:
        if height[left] < height[right]:
            left_max = max(left_max, height[left])
            water += left_max - height[left]
            left += 1
        else:
            right_max = max(right_max, height[right])
            water += right_max - height[right]
            right -= 1
    return water
```

Time: O(n) **Space:** O(1)

Follow-ups

- Compare with the prefix-max array (O(n) space) approach.
- Solve it with a monotonic stack instead.
- Extend to the 2D "trapping rain water" (matrix) version.
- Return water trapped above each index, not just the total.

6. Remove Duplicates from Sorted Array

Pointers: slow, fast (same dir.) **Invalid:** nums[fast]==nums[fast-1]

```
def removeDuplicates(nums):
    if not nums:
        return 0
    slow = 1
    for fast in range(1, len(nums)):
        if nums[fast] != nums[fast - 1]:
            nums[slow] = nums[fast]
            slow += 1
    return slow
```

Time: O(n) **Space:** O(1)

Follow-ups

- Allow each element to appear at most twice.
- What if the array isn't sorted?
- Do it without mutating the input array.
- Generalize to "at most k duplicates allowed."

7. Move Zeroes

Pointers: slow, fast (same dir.) **Action:** swap nonzero forward

```
def moveZeroes(nums):
    slow = 0
    for fast in range(len(nums)):
        if nums[fast] != 0:
            nums[slow], nums[fast] = nums[fast],
            nums[slow]
            slow += 1
```

Time: O(n) **Space:** O(1)

Follow-ups

- Why does this already minimize the number of swaps?
- Move all zeroes to the front instead of the back.
- Generalize to move any given value to the end.
- Does relative order of non-zero elements matter here?

8. Sort Colors

Pointers: low, mid, high **Rule:** Dutch National Flag

```
def sortColors(nums):
    low, mid, high = 0, 0, len(nums) - 1
    while mid <= high:
        if nums[mid] == 0:
            nums[low], nums[mid] = nums[mid],
            nums[low]
            low += 1
            mid += 1
        elif nums[mid] == 1:
            mid += 1
        else:
            nums[mid], nums[high] = nums[high],
            nums[mid]
            high -= 1
    return nums
```

Time: O(n) **Space:** O(1)

Follow-ups

- Why isn't mid incremented after swapping with high?
- Generalize to k colors (k-way partition).
- Compare with counting sort — trade-offs?
- Make it a stable partition (preserve relative order).

Two Pointers Cheat Sheet — Part 2 (3Sum Family)

Longer two-pointer solutions built on a sort + fix-one-index-then-two-pointer pattern

9. 3Sum

Pattern: sort + fix i + 2 pointers **Skip:** duplicate i, left, right

```
def threeSum(nums):
    nums.sort()
    res = []
    for i in range(len(nums) - 2):
        if i > 0 and nums[i] == nums[i - 1]:
            continue
        left, right = i + 1, len(nums) - 1
        while left < right:
            s = nums[i] + nums[left] + nums[right]
            if s == 0:
                res.append([nums[i], nums[left], nums[right]])
                left += 1
                right -= 1
                while left < right and nums[left] == nums[left - 1]:
                    left += 1
                while left < right and nums[right] == nums[right + 1]:
                    right -= 1
            elif s < 0:
                left += 1
            else:
                right -= 1
    return res
```

Time: $O(n^2)$ **Space:** $O(\log n) - O(n)$ for sort (excl. output)

Follow-ups

- Generalize to 4Sum / general k-Sum.
- Return original indices instead of values (sorting changes them).
- Count the number of triplets instead of listing them.
- If nums has no duplicates guaranteed, does the dedup logic simplify?

10. 3Sum Closest

Pattern: sort + fix i + 2 pointers **Track:** min |sum - target|

```
def threeSumClosest(nums, target):
    nums.sort()
    closest = nums[0] + nums[1] + nums[2]
    for i in range(len(nums) - 2):
        left, right = i + 1, len(nums) - 1
        while left < right:
            s = nums[i] + nums[left] + nums[right]
            if abs(s - target) < abs(closest - target):
                closest = s
            if s == target:
                return s
            elif s < target:
                left += 1
            else:
                right -= 1
    return closest
```

Time: $O(n^2)$ **Space:** $O(\log n) - O(n)$ for sort

Follow-ups

- Return the actual triplet, not just the closest sum.
- Generalize to k numbers closest to target.
- What if multiple triplets tie for closest — return all of them?
- Add duplicate-skipping like 3Sum — does it change the result?

Complexity Summary

#	Problem	Time	Space
1	Two Sum II	$O(n)$	$O(1)$
2	Valid Palindrome	$O(n)$	$O(1)$
3	Valid Palindrome II	$O(n)$	$O(1)$
4	Container With Most Water	$O(n)$	$O(1)$
5	Trapping Rain Water	$O(n)$	$O(1)$
6	Remove Duplicates	$O(n)$	$O(1)$
7	Move Zeroes	$O(n)$	$O(1)$
8	Sort Colors	$O(n)$	$O(1)$
9	3Sum	$O(n^2)$	$O(\log n) - O(n)$
10	3Sum Closest	$O(n^2)$	$O(\log n) - O(n)$

General Two-Pointer Tips

Opposite-direction pointers (left/right ends, closing inward) suit sorted arrays and problems comparing a pair against a target — Two Sum II, palindromes, Container With Most Water.

Same-direction pointers (slow/fast) suit in-place array rewriting — Remove Duplicates, Move Zeroes.

Sort first whenever order isn't given but comparisons against a target are needed (3Sum family) — sorting costs $O(n \log n)$ but unlocks $O(1)$ pointer movement instead of nested search.

Dedup pattern: after processing a value, skip forward/backward past any equal neighbors to avoid duplicate results.

Follow-Up Answers (Part 1 of 3)

Simple explanations for the follow-up questions on problems 1–4

1. Two Sum II (Sorted Array)

Unsorted array version?

Two-pointer needs sorted order to know which direction to move. Unsorted, use a hash map instead: for each number, check if (target – number) was already seen — $O(n)$ time, $O(n)$ space instead of $O(1)$ space.

Return all pairs, skipping duplicates?

After a match, record it, move both pointers inward, then skip over any values equal to the one just used — same dedup pattern as 3Sum.

Need the lexicographically smallest pair?

The two-pointer scan naturally finds the pair with the smallest left index first, so the first match found is already the answer — just return immediately.

Generalize to k-Sum?

Recursively fix one number at a time, reducing to a $(k-1)$ -Sum problem, bottoming out at Two Sum (two-pointer) when $k = 2$.

2. Valid Palindrome

Ignore only spaces, keep punctuation?

Change the skip condition from "not isalnum()" to "==" ' ' " (or whichever characters to ignore), and compare characters directly.

Return the cleaned/filtered string too?

Build a filtered string in a first pass (keep only alnum, lowercase), then check it directly or run two-pointer on it while also returning it.

Handle a streaming input?

Two-pointer from both ends needs random access to the end, so a true stream needs buffering first (loses the streaming benefit) — or restructure using a stack/deque while reading forward.

Make the alnum check Unicode-aware?

Python's str.isalnum() already handles Unicode letters and digits correctly — just watch for locale-specific casing rules if you manually lowercase.

3. Valid Palindrome II

Generalize to "at most k deletions"?

The simple two-branch recursion becomes exponential for general k . Use dynamic programming instead — the "minimum deletions to make a palindrome" DP — and check if that minimum is $\leq k$.

Return the resulting palindrome string?

When you recurse into isPal(left+1,right) or isPal(left,right-1), remember which branch succeeded — that tells you which character to delete — then remove it from the original string.

Count the number of ways with one deletion?

At the first mismatch, check both branches independently (don't short-circuit with "or") and count how many of the two succeed.

What if deletions are only from the ends?

Just delete straight from left+=1 or right-=1 in the outer loop itself when a mismatch occurs, instead of trying an inner substring check.

4. Container With Most Water

Why is moving the shorter wall always safe?

Area is capped by the shorter wall. Keeping it and moving the other pointer only shrinks the width with the same limiting height — never better. Moving the shorter wall is the only move that can find a taller limiting wall.

Return the indices of the best container?

Track best_left and best_right alongside best area, updating all three together whenever a new max is found.

Find the top-3 largest areas?

Greedy two-pointer only safely finds the global max. For top-k you'd need a min-heap of size k while scanning, since greedy pointer movement discards candidates you might need later.

Extend to 3 lines — still optimal greedily?

No — the "discard the worse side" argument breaks down because the optimal triplet isn't guaranteed to include an extreme pointer. Needs a different approach entirely.

Follow-Up Answers (Part 2 of 3)

Simple explanations for the follow-up questions on problems 5–8

5. Trapping Rain Water

Compare with the prefix-max array approach?

Precompute `left_max[i]` and `right_max[i]` in two passes, then `water[i] = min(left_max[i], right_max[i]) - height[i]`. Same $O(n)$ time but $O(n)$ extra space, versus $O(1)$ for two-pointer.

Solve with a monotonic stack?

Push indices while heights are decreasing; when a taller bar appears, pop and compute water trapped between the new stack top and the current bar, layer by layer. Also $O(n)$, and gives water "basin by basin."

Extend to the 2D matrix version?

Use a min-heap starting from the boundary cells, always expanding from the lowest boundary outward — similar to Dijkstra. A different technique from simple two-pointer.

Return water trapped above each index?

Store `left_max - height[i]` (or `right_max - height[i]`, whichever side processed that index) into a `water[]` array instead of just accumulating a running total.

6. Remove Duplicates from Sorted Array

Allow each element up to twice?

Compare `nums[fast]` to `nums[slow-2]` instead of `nums[fast-1]`; only skip the write if it would create a third consecutive duplicate.

What if the array isn't sorted?

Sorted order is what makes duplicates adjacent. Unsorted, you'd need a hash set to track seen values — $O(n)$ extra space instead of $O(1)$.

Do it without mutating the input?

Build and return a new list instead of writing into `nums` in place — same logic, just append rather than overwrite.

Generalize to "at most k duplicates"?

Compare `nums[fast]` to `nums[slow-k]` and only skip the write if that would exceed k consecutive duplicates.

7. Move Zeroes

Why does this minimize swaps already?

A swap only happens when a non-zero is found ahead of a zero, and each swap places exactly one non-zero in its final position — so swap count equals the minimum number of out-of-place elements.

Move zeroes to the front instead?

Flip the comparison — scan for zeros instead of non-zeros with the fast pointer, and swap them toward the front.

Generalize to move any value to the end?

Replace the `"!= 0"` check with `"!= value"` — the rest of the slow/fast swap pattern is unchanged.

Does relative order of non-zeros matter?

Yes, and this solution preserves it — slow never passes fast, and each swap places the next non-zero in order, so relative order among non-zero elements is kept.

8. Sort Colors

Why isn't mid incremented after swapping with high?

The value swapped in from `nums[high]` hasn't been examined yet (could be 0, 1, or 2), so mid must re-check it. A value swapped in from `nums[low]` is already known to be ≤ 1 , so mid can safely advance there.

Generalize to k colors?

The three-pointer trick doesn't extend past 3 buckets directly. Use counting sort instead: count each color in one pass, then overwrite the array in a second pass.

Compare with counting sort?

Counting sort is $O(n+k)$ and easy to reason about but needs two passes. Dutch flag does it in one pass, useful when you can only look at each element once (e.g., streaming).

Make it a stable partition?

The in-place swap approach isn't stable. For stability, write elements of each color into three separate lists in original order, then concatenate.

Follow-Up Answers (Part 3 of 3)

Simple explanations for the follow-up questions on problems 9–10 (3Sum family)

9. 3Sum

Generalize to 4Sum / general k-Sum?

Add another outer loop to fix a second index before the two-pointer scan; more generally, recursively fix one number at a time until two remain, then two-pointer — the standard k-Sum reduction.

Return original indices instead of values?

Before sorting, pair each value with its original index (sort a list of (value, index) tuples), keeping the index alongside for the final answer.

Count triplets instead of listing them?

Increment a counter instead of appending to a result list — the skip-duplicate logic still ensures each unique triplet is counted exactly once.

No duplicates guaranteed — does dedup simplify?

Yes — you can drop the "skip duplicate value" checks for i, left, and right entirely, since every combination found is automatically unique. Saves constant work, same $O(n^2)$.

10. 3Sum Closest

Return the actual triplet, not just the sum?

Track the three values (or indices) alongside "closest," updating all of them together whenever a new closest sum is found.

Generalize to k numbers closest to target?

Same recursive reduction as k-Sum — fix numbers one at a time, track the closest combination at the base two-pointer case, and propagate the best result back up through the recursion.

Multiple triplets tie for closest — return all?

After finding the minimum $|\text{sum} - \text{target}|$, do a second pass (or collect during the first) recording every triplet achieving that exact difference, using the same duplicate-skipping as 3Sum.

Add duplicate-skipping like 3Sum — does it change the result?

The closest sum itself won't change, but skipping duplicates avoids wasted redundant comparisons when many equal values are adjacent — meaningfully faster on inputs with lots of repeats.